LEVEL - 4

1. Circular prime number (The number when rotated will also be prime number)

```
def func(n):
    for i in n:
        s = str(i)
        f = 1
        for j in range(len(s)):
            rot = int(s[j:] + s[:j])
            if rot < 2:
                f = 0
                break
            for k in range(2, int(rot**0.5) + 1):
                if rot % k == 0:
                    f = 0
                    break
            if f == 0:
                break
        if f == 0:
            print("NOT")
        else:
            print("YES")

n = input().split()
func(n)
```

2. ATM MACHINE

User inputs: "5 10" (10 - TOTAL AMOUNT AVAILABLE IN ATM)
User inputs: "3 5 7 2 1" (IF WITHDRAW POSSIBLE - 1 , ELSE - 0)
Outputs: "11010"

```
def func(n, nums):
    amt = int(n[1])
    lst = []
```

```
    ans = ""
    for i in nums:
        lst.append(int(i))
    for i in lst:
        if amt >= i :
            amt -= i
            ans += "1"
        else:
            ans += "0"
    print(ans)

n = input().split()
nums = input().split()
func(n,nums)
```

3. MULTIPLY THE VALUES IN LIST (if there exist a float value then the final answer must be in float)

Input:	output :
2,3,4,5	120
1.5,2,3.5,4	46.0

```
def func(n):
    lst =[]
    c = 0
    s= str(n)
    if '.' in s:
        c = 1
    for i in n:
        lst.append(float(i))
    ans =1
    for i in lst:
        ans *= i
    if c==1:
        print(float(ans))
    else:
        print(int(ans))
```

```
n = input().split(',')
func(n)
```

4. BUY TWO CHOCOLATES

INPUT1:	input2;	output:
1,2,2	3	0
3,5,4	6	6

```
def func(n,amt):
    lst = []
    for i in n:
        lst.append(int(i))
    lst.sort()
    if lst[0] + lst[1] <= amt:
        amt -= lst[0]
        amt -= lst[1]
    print(amt)

n = input().split()
amt = int(input())
func(n,amt)
```

5. MAXIMUM PRODUCT OF THREE NUMBERS

Input;	output:
1,2,3,4	24

```
def func(n):
    lst = []
    for i in n:
        lst.append(int(i))
    lst.sort()
    m1 = lst[-1]*lst[-2]*lst[-3]
    m2 = lst[0]*lst[1]*lst[-1]
```

```
    ans = max(m1,m2)
    print(ans)

n = input().split(',')
func(n)
```

6. COUNT OF PALINDROMIC SUBSTRING

Input: aba **output** : 4

Substrings checked:

- "a" → palindrome ✔️
- "ab" → ❌
- "aba" → ✔️
- "b" → ✔️
- "ba" → ❌
- "a" → ✔️

✅ Total palindromic substrings: 4

```
def func(s):
    c = 0

    for i in range(len(s)):
        for j in range(i+1, len(s)+1):
            substr = s[i:j]
            if substr == substr[::-1]: # check palindrome
                c += 1
    print(c)

s = input()
func(s)
```

7. COMBINATIONS

C(n,r)=n! / r!(n−r)!

Input: output:

5 10

2

```
import math
def func(n,r):
    print(math.comb(n,r))

n = int(input())
r = int(input())
func(n, r)
```

8. PRIME NUMBERS IN A GIVEN RANGE

Input: output:

10 11 13 17 19

20

```
def func(n1, n2):
    for i in range(n1, n2):
        f = 1
        for j in range(2,i):
            if i % j == 0:
                f = 0
                break
        if f == 1 and i > 1:
            print(i, end = ' ')

n1 = int(input())
n2 = int(input())
func(n1, n2)
```

9. MAXIMUM DIGIT SUM OF TWO NUMBERS

Input: output:
3 12
8 6 7

(8,6) (8,7) (6,7) => 8*6=48, 8*7= 56, 6*7 = 42
Current sum(csum) : 4+8 =12, 5+6 = 11, 4+2=6;
max = 12

```
def func(n, nums):
    lst = []
    m = 0
    for i in nums:
        lst.append(int(i))
    for i in range(len(lst)):
        for j in range(i+1, len(lst)):
            p = lst[i]*lst[j]
            csum = sum(int(k) for k in str(p))
            m = max(m, csum)
    print(m)

n = int(input())
nums = input().split()
func(n, nums)
```

10. TIME CONVERTION
Enter time in 12-hour format (e.g., 07:05:45PM):

OUTPUT:
19:05:45

```
def func(s):
    hr = int(s[:2])     #  07
    ms = s[2:8]      #  :05:45
```

```
    ap = s[-2:]     #  PM

    if ap =='AM':
        if hr == 12:
            hr = 0
    else: #ap == 'PM'
        if hr != 12:
            hr += 12
    print("{:02d}{}".format(hr, ms))

s = input()
func(s)
```

11. UPPER AND LOWER LETTERS COUNT

```
def func(s):
    u = 0
    l = 0
    for i in s:
        if i.isupper():
            u += 1
        if i.islower():
            l += 1
    print("UPPER: ",u, "LOWER: ",l)

s = input()
func(s)
```

12. KEYBOARD ROW WORD CHECK

INPUT:	**OUTPUT:**
5	
Hello	**['Alaska', 'Dad', 'Qwerty']**
Alaska	
Dad	
Peace	
Qwerty	

```
def func(s):
    ans=[]
    for i in s:
        a=0
        b=0
        c=0
        l = len(i)
        low = i.lower()
        for j in low :
            if j in 'qwertyuiop':
                a =a+1
            if j in 'asdfghjkl':
                b =b+1
            if j in 'zxcvbnm':
                c =c+1
        if a==l or b==l or c==l:
            ans.append(i)
    print(ans)

n = int(input())
s=[]
for i in range(0, n):
    s.append(input())
func(s)
```

13. MAXIMUM PRODUCT OF WORD LENGTH (when there are no common letters)

Input:	**output:**
5	**30**
hello	
world	
python	
java	
script	

```
def func(s,n):
    maxp = 0
    for i in range(0, n):
        l1 = len(s[i])
        for j in range(i+1, n):
            l2 = len(s[j])
            if not set(s[i]) & set(s[j]):
                cp = l1*l2
                maxp = max(maxp, cp)
    print(maxp)

n = int(input())
s = []
for i in range(0,n):
    s.append(input())
func(s,n)
```

—---or

```
def func(lst):
    m = 0
    for i in range(len(lst)):
        for j in range(i+1, len(lst)):
            if not set(lst[i]) & set(lst[j]):
                p = len(lst[i]) * len(lst[j])
                if p > m:
                    m = p
    print(m)

n = int(input())
lst = []
for _ in range(n):
    lst.append(input())
func(lst)
```

14. INTEGER TO ENGLISH WORDS

15. DECIMAL TO BINARY WITH 7 PRECISION

Input:	**output:**
10.625	**1010.1010000**

```
def func(n):
    ipart = int(n)
    fpart = n - ipart

    ibin = bin(ipart)[2:]
    fbin = '.'

    for i in range(0, 7):
        fpart *= 2
        b = int(fpart)
        fbin += str(b)
        fpart -= b
    print(ibin + fbin)

n = float(input())
func(n)
```

16. PRINT THE NUMBERS FOUND AT LEAST IN TWO SETS (answer must be in list)

Input:	**output:**
1,2,3,4	**[3, 4, 5, 6]**
3,4,5,6	
4,5,6,7	

```
def func(n1, n2, n3):
    num1, num2, num3 = [], [], []
    for i in n1:
        num1.append(int(i))
    for i in n2:
        num2.append(int(i))
```

```
    for i in n3:
        num3.append(int(i))
    s1, s2, s3 = set(num1), set(num2), set(num3)
    ans = (s1&s2) | (s1&s3) | (s2&s3)
    lst = list(ans)
    lst.sort
    print(lst)

n1 = input().split(',')
n2 = input().split(',')
n3 = input().split(',')
func(n1,n2,n3)
```

17. LONGEST WORD IN A SENTENCE

INPUT:	OUTPUT:
I love programming in Python	programming

```
def func(s):
    w = s.split()
    ans = max(w, key=len)
    print(ans)

s = input()
func(s)
```

18. DECIMAL TO BINARY

```
def fun(n):
    bi = bin(int(n))[2:]
    print(bi)
```

```
n = float(input())
fun(n)
```

19. PALINDROME

```
def func(s):
    s = s.replace(" ", "").lower()
    if s == s[::-1]:
        print("Palindrome")
    else:
        print("NotPalindrome")

s = input()
func(s)
```

20.House Robber program

```
def rob_houses(money, i, memo):
    if i >= len(money):
        return 0
    if i in memo:
        return memo[i]

    # Rob this house or skip it
    rob = money[i] + rob_houses(money, i + 2, memo)
    skip = rob_houses(money, i + 1, memo)

    memo[i] = max(rob, skip)
    return memo[i]

# Get input from user
nums = list(map(int, input("Enter house money separated by spaces: ").split()))

# Call the function
result = rob_houses(nums, 0, {})
print("Maximum money that can be robbed:", result)
```

21. FIBONACCI SERIES

```
def fibonacci(n):
    if n == 0:
        return []
    elif n == 1:
        return [0]

    lst= [0, 1]
    for i in range(2, n):  # no recursion using loop & list
        lst.append(lst[i-1] + lst[i-2])
    print(lst)

n = int(input("Enter number of terms: "))
fibonacci(n)  # End of program and output in list with [] & ,

—-to print with space - neglecting comma
lst = fibonacci(n)

for i in lst :
     print (i, end=' ')
—-
```

22. REPLACE A CHARACTER

```
def func(s, o, n):
    return s.replace(o, n)

s = input()
o = input()
n = input()
print(func(s, o, n))
```

23.Generate Tuples of Numbers and Their Squares in a Range

Input 1: 2 4 Output 1: [(2,4),(3,9),(4,16)]

Input 2: 101 99 Output 2: Invalid Range
(Explanation:First number greater than second so invalid)
Input 3: 101 101 Output 3: Equal range

```
def func(s,e):
    if s< e:
        lst =[]
        for i in range(s, e+1):
            lst.append((i, i**2)) # power
        print(lst)
    else:
        print("Invalid")

s, e = map(int,input().split())
func(s, e)
```

24. Print All Integers from N to -1
input:-5 output:-4 -3 -2 -1

```
def func(n):
    if n >=-1:
        print("Invalid")
    else:
        for i in range(n+1, 0):
            print(i, end=' ')

n = int(input())
func(n)
```

24. LONGEST WORD IN A STRING

```
def func(s):
    ans=""
    maxi=0
    for i in s:
        l = len(i);
        if(l>maxi):
            maxi = l
            ans = i
    print(ans)

s = input().split()
func(s)
```

25. DECIMAL TO BINARY

INPUT: 25.75 OUTPUT: 11001.11

INPUT: 15 OUTPUT: 1111

```
def func(n):
    ipart = int(n)
    fpart = n - ipart

    ibin = bin(ipart)[2:]
    fbin = '.'

    for i in range(0, 2):
        fpart *= 2
        b = int(fpart)
        fbin += str(b)
        fpart -= b

    if n-ipart == 0:  # if input is integer then print without deci
        print(ibin )
    else:             # if input is float then print with 2 precision
        print(ibin+ fbin)

n = float(input())
```

func(n)

26. Return the largest lucky integer in the array.

If there is no lucky integer return -1.

—----------

Example 1:
Input: arr = [2,2,3,4]
Output: 2
Explanation: The only lucky number in the array is 2 because frequency[2] == 2.

Example 2:
Input: arr = [1,2,2,3,3,3]
Output: 3
Explanation: 1, 2 and 3 are all lucky numbers, return the largest of them.

Example 3:
Input: arr = [2,2,2,3,3]
Output: -1
Explanation: There are no lucky numbers in the array.

—----------

```
def func(n):
    lst =[]
    for i in n:
        val = int(i)
        c = n.count(i)
        if c == val:
            lst.append(val)
    m = -1
    for i in lst:
        if int(i) > m:
            m = int(i)
    print(m)

n = input().split(',')
func(n)
```

27. Interleave Strings using recursive functions

ip1:ab ip2:cd
op:abcd acbd acdb cabd cadb cdab

```
def interleave(s1, s2, res=""):
    if not s1 and not s2:
        print(res)
        return
    if s1:
        interleave(s1[1:], s2, res+s1[0])
    if s2:
        interleave(s1, s2[1:], res+s2[0])

s1 = input()
s2 = input()
interleave(s1, s2)
```

28. Write a program that takes an integer N as input and returns a list of its prime factors, where each prime factor appears as many times as it divides N.

Sample 1:
Input:
>>> 56 Output:[2, 2, 2, 7]

Non-prime numbers are made up of smaller primes that were already removed earlier.

```
def func(n):
    lst=[]
    i =2
    while(i <= n):
```

```
        if n % i==0:
            lst.append(i)
            n = n//i
        else:
            i+=1
    print(lst)

num = int(input())
func(num)
```

29. a permutation of a string

Input: "abc"
Output: ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']

In-built

```
from itertools import permutations

def string_permutations(s):
    result = [''.join(p) for p in permutations(s)]
    print(result)

s= input()
string_permutations(s)
```

—----------**Manual**—----------

```
def permute(s, answer="", result=None):
    if result is None:
        result = []

    if len(s) == 0:
        result.append(answer)
        return result

    for i in range(len(s)):
        ch = s[i]
```

```
        left = s[:i]
        right = s[i+1:]
        rest = left + right
        permute(rest, answer + ch, result)

    return result

# Example usage
string = input("Enter a string: ")
output = permute(string)
print(output)
```

30. Middle digit of a number

Input: 12345 output: 3
Input: 1234 output: 2

```
def func(n):
    mid = 0
    s = str(n)
    for i in range(0,len(s)):
        if len(s) % 2 !=0: # odd - len of number
            if i == len(s)//2:
                mid = int(s[i])
                break
        else:  # even - len of number
            if i == len(s)//2-1:
                mid = int(s[i])
                break
    print(mid)

n = int(input())
func(n)
```

31. print prime,composite number and print negative numbers in sorted order

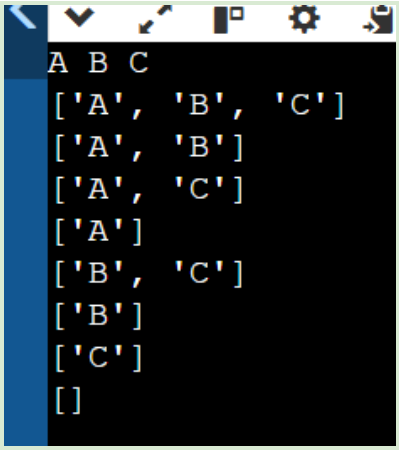
i/t: 2 3 4 5 -3 -2 -7
o/t:
[2,3,5]
[4]
[-2,-3,-7]

```
def func(n):
    lst1 = []  # Prime numbers
    lst2 = []  # Non-prime positive numbers
    lst3 = []  # Negative numbers

    for i in n:
        num = int(i)

        if num < 0:
            lst3.append(num)
        elif num <= 1:
            lst2.append(num)
        else:
            is_prime = True
            for j in range(2, int(num**0.5) + 1):
                if num % j == 0:
                    is_prime = False
                    break
            if is_prime:
                lst1.append(num)
            else:
                lst2.append(num)

    print(lst1)
    print(lst2)
    print(lst3)

n = input().split()
func(n)
```

32. Similar Letters

```
from collections import Counter

w="god"
v="dog"
if Counter(w)==Counter(v):
    print("yes")
else:
    print("No")
```

33. RECURSIVE SUBSTRING GENERATOR




```
def func(s, index=0, curr=[]):
    if index == len(s):
        print(curr)
        return
    func(s, index+1, curr+[s[index]]) # square bracket
    func(s, index+1, curr)

s=input().split()
func(s)
```

34. INTEGER TO STRING

https://leetcode.com/problems/integer-to-english-words/description/

```
def numberToWords(n): # 1 - 19, 10 - 90, 1000-M-B
    to19 = ['','One','Two','Three','Four','Five','Six','Seven','Eight','Nine','Ten',
```

```
        'Eleven','Twelve','Thirteen','Fourteen','Fifteen','Sixteen','Seventeen',
        'Eighteen','Nineteen']
    tens = ['','Ten','Twenty','Thirty','Forty','Fifty','Sixty','Seventy','Eighty','Ninety']
    units = ['','Thousand','Million','Billion']

    def word(num):
        if num == 0: return ''
        if num < 20: return to19[num] + ' '
        if num < 100: return tens[num//10] + ' ' + word(num%10) # paranthesis
        return to19[num//100] + ' Hundred ' + word(num%100)

    if n == 0: return 'Zero'

    res = ''
    i = 0

    while n > 0:
        if n % 1000 != 0:
            res = word(n % 1000) + units[i] + ' ' + res
        n //= 1000
        i += 1
    return res.strip() # to remove extra space

num = int(input("Enter a number: "))
print(numberToWords(num))
```

35. THREE CONSECUTIVE CONSONANTS

```
def func(s):
    vowels = 'aeiouAEIOU'
    count = 0
    ans =""
    for ch in s:
        if ch.isalpha() and ch not in vowels:
            count+=1
            if count == 3:
                ans += "Yes"
```

```
            break
        else:
            count =0

    if count != 3:
        ans +="No"

    print(ans)

s = input()
func(s)
```

36. Two out of three

- https://leetcode.com/problems/two-out-of-three/description/

Answer in any order

Input: nums1 = [1,1,3,2], nums2 = [2,3], nums3 = [3]

Output: [3,2]

```
def func(n1, n2, n3):
    lst=[]
    for i in n1:
        for j in n2:
            if i == j:
                lst.append(i)
    for i in n1:
        for j in n3:
            if i == j:
                lst.append(i)
    for i in n2:
        for j in n3:
            if i == j:
                lst.append(i)
    lst = list(set(lst))
    if lst:
        return lst
    else:
        return []
```

```
n1 = list(map(int, input().split(',')))
n2 = list(map(int, input().split(',')))
n3 = list(map(int, input().split(',')))
print(func(n1, n2, n3))
```

37. Last stone weight

-https://leetcode.com/problems/last-stone-weight/description/

```
def func(stones):
    while len(stones) > 1:
        stones.sort()
        max1 = stones.pop()
        max2 = stones.pop()
        if max1 != max2:
            stones.append(max1 - max2)

    if stones:
        return stones[0]
    else:
        return 0

stones = list(map(int, input().split(',')))
print(func(stones))
```

38.Min cost climbing stairs

-https://leetcode.com/problems/min-cost-climbing-stairs/description/

```
def func(cost):
    n = len(cost)
    if n == 0:
```

```
        return 0
    elif n == 1:
        return cost[0]
    else:
        a, b = cost[0], cost [1]
        for i in range(2, n):
            a, b = b, cost[i] + min(a, b)

        return min(a,b)

cost = list(map(int, input().split()))
print(func(cost))
```

39. Quadratic Equation Solver

```
import math
def func(a,b,c):
    d = b**2 - 4*a*c

    if d > 0:
        r1 = (-b + math.sqrt(d)) / (2*a)
        r2 = (-b - math.sqrt(d)) / (2*a)
        print("Root 1: ", r1, " Root 2: ", r2)

    elif d == 0:
        r = (-b) / (2*a)
        print("One root: ", r)

    else:
        print("No real root exist")

a = float(input())
b = float(input())
c = float(input())
func(a,b,c)
```

40. Intersection of two array
https://leetcode.com/problems/intersection-of-two-arrays/description/

Example 1:
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2]

Example 2:
Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [9,4]
Explanation: [4,9] is also accepted.

```
def func(n1, n2):
    lst=[]
    for i in n1:
        for j in n2:
            if int(i) == int(j) :
                lst.append(int(i))
                break
    new_list = list(set(lst))
    print(new_list)
n1 = input().split(',')
n2 = input().split(',')
func(n1, n2)
```

41. Poisoned Duration Get set of stones in list and Duration in int
i/p:[2,4,6] 2 o/p:7
https://leetcode.com/problems/teemo-attacking/

```
def poison(lst, duration):
    if not lst:
        return 0
```

```
    total = 0
    for i in range(len(lst) - 1):
        total += min(lst[i+1] - lst[i], duration)

    total += duration
    return total

lst = list(map(int, input().split(',')))
duration = int(input())

print(poison(lst, duration))
```

42. printing the area of circle,triangle and rectangle for finding area of circle import math

```
import math
def area_circle(radius):
    return math.pi * radius * radius

def area_triangle(base, height):
    return 0.5 * base * height

def area_rectangle(length, width):
    return length * width

r = float(input("Enter radius of circle: "))
b = float(input("Enter base of triangle: "))
h = float(input("Enter height of triangle: "))
l = float(input("Enter length of rectangle: "))
w = float(input("Enter width of rectangle: "))

print(f"{area_circle(r):.2f}")
print(f"{area_triangle(b, h):.2f}")
print(f"{area_rectangle(l, w):.2f}")
```

43. DEPTH OF DICTIONARY

```
def dep_dict(d):
    if not isinstance(d, dict) or not d:
        return 0
    return 1 + max(dep_dict(v) for v in d.values())

s = input()
n_dict = eval(s)
print(dep_dict(n_dict))
```

—----------

Minimum Amount of Time to Fill Cups - LeetCode

MINIMUM ELEMENT SHARED BY BOTH ARRAYS

Last visited integer -
https://leetcode.com/problems/last-visited-integers/description/

count the depth of the nested dictionary